

Enhancement delivery — what you are looking at

The scenario, the insurer, and the application. Two minutes of ground before anything moves.

THE SCENARIO

S3 · Enhancement delivery

A small, ordinary change to an application that **is already live** — the kind a team picks up most weeks. We follow one end to end: a written request on a board, out to a released change.

Five stages, and **a person signs off at every one**. This is not a slide — the applications are running, and the change is really made to their code.

THE ESTATE

MapleSure Insurance

A made-up group benefits insurer. **Every application, contract, plan sponsor and person here is invented** — no client code and no client data anywhere.

What is not invented is the shape: applications owned by different teams, with the same hand-offs you have. Most of what goes wrong in a change like this goes wrong **between** applications, not inside one.

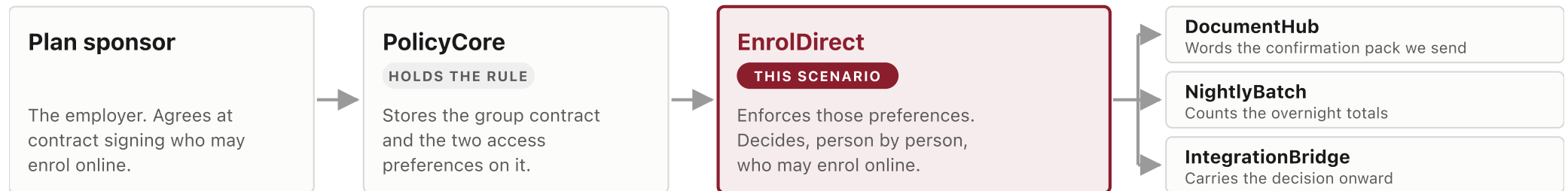
THE APPLICATION

EnrolDirect

MapleSure's online enrolment channel. **Its user is the plan member** — an employee joining a benefit plan themselves, on the web, instead of phoning the service centre.

Every one of them meets the same question: **is this person allowed to enrol online?** EnrolDirect does not own that rule — the sponsor agrees it, PolicyCore stores it, EnrolDirect enforces it, and three other systems read the answer.

WHERE THE RULE COMES FROM



WHO USES THE ANSWER

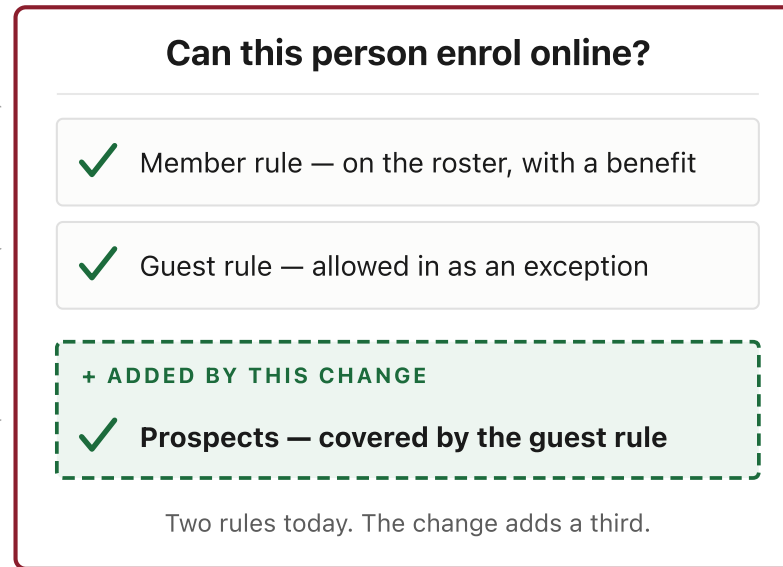
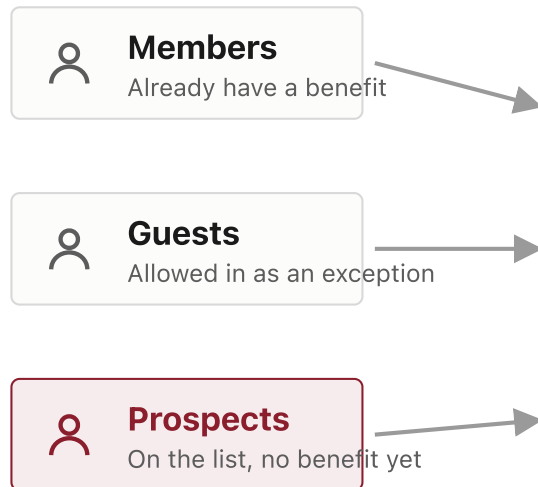
Nothing here is a technical decision. **A plan sponsor's rule, enforced by one application and read by three others.** What follows is what happens when that rule has to cover a group of people nobody ever wrote a rule for.

Letting prospect members enrol online

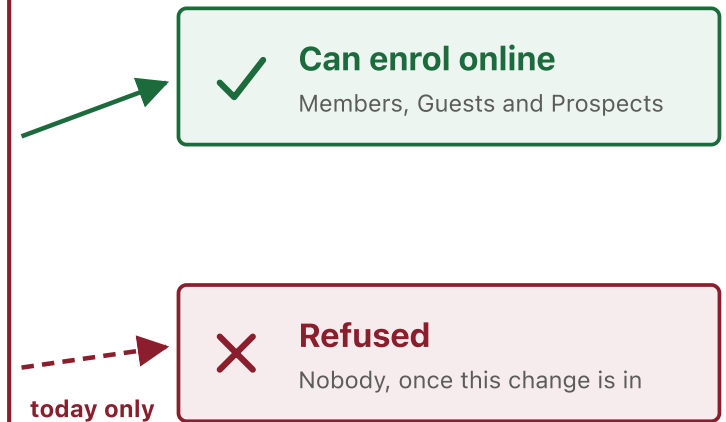
What the business asked for, and what AI will do about it — before we open any tools.

EnrolDirect is where people join their benefit plan online, without phoning anyone. A single check decides who is allowed in.

WHO TURNS UP



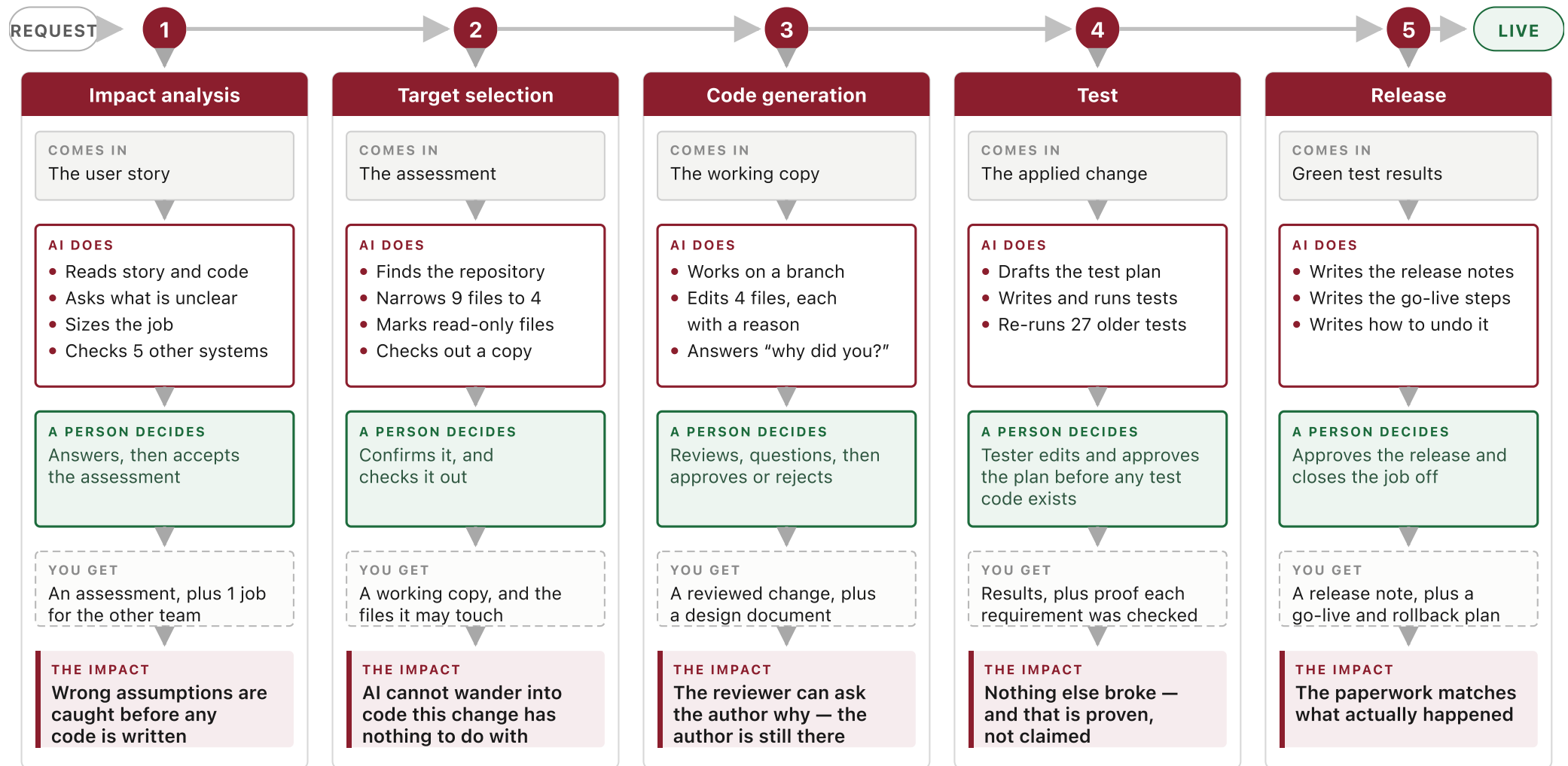
THE ANSWER



That is the whole change: **we add one rule, and one group of people gets a yes instead of a no.** It arrives as a written request on the delivery team's board.

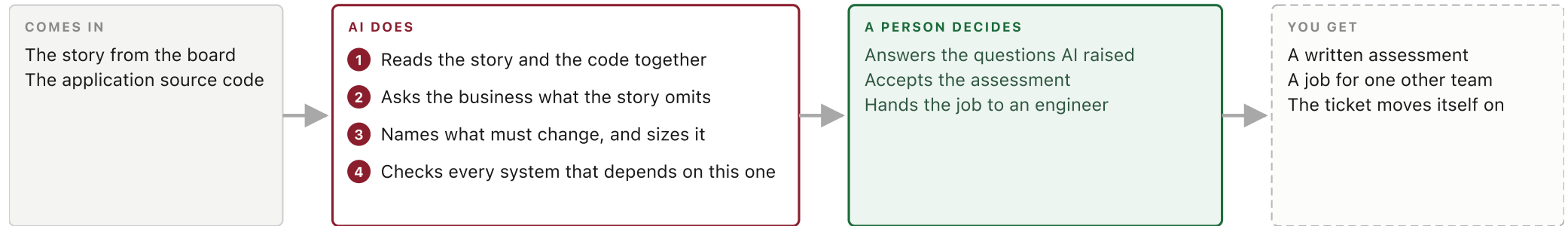
What AI does — and where a person still decides

The request comes in on the left and goes live on the right. **AI helps at all five stages. A person signs off at all five.** The work does not move on until they do — and each stage has its own page after this one.

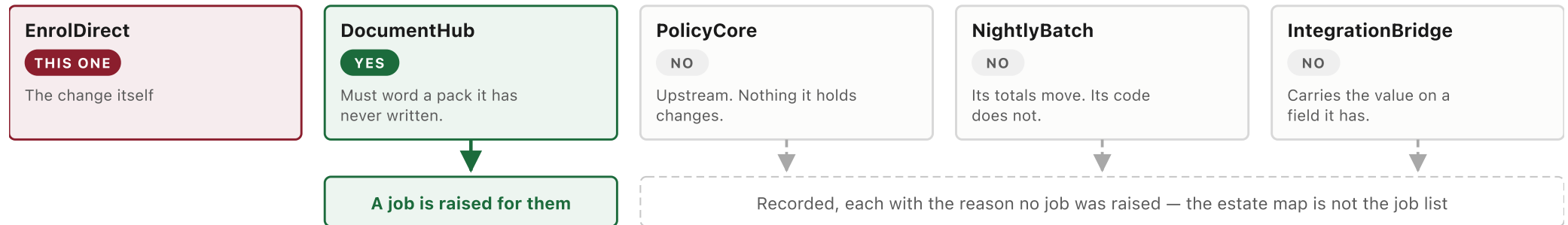


Stage 1 · Impact analysis

What has to change, how big it is, and who else it touches — worked out before anyone opens an editor.



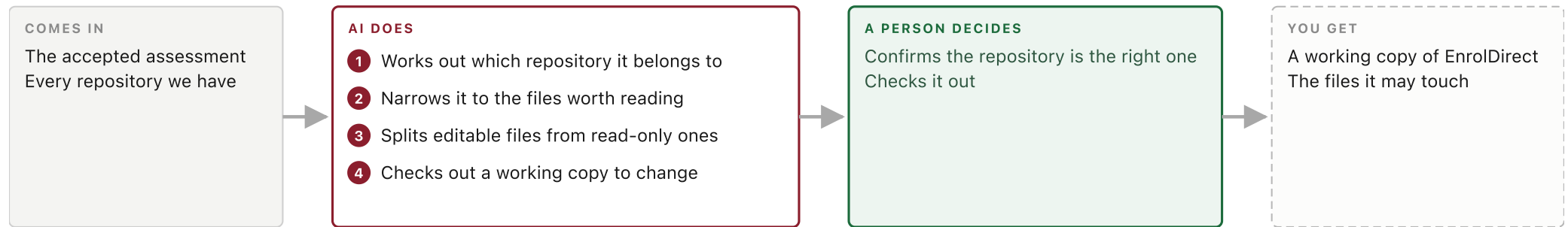
How it decides whether another team is actually affected



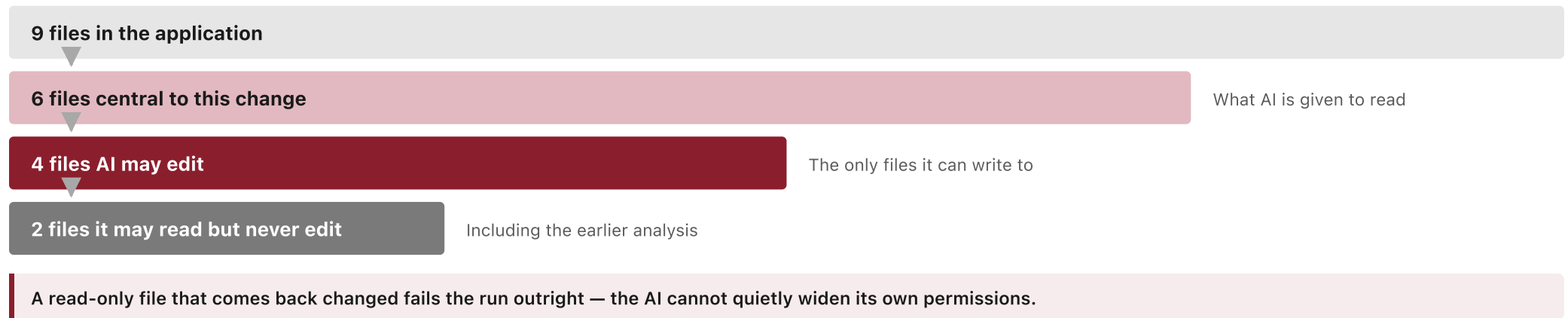
The bar is “that team must change code”, not “that team is affected”. Three of these are affected and have nothing to do — raising jobs for them wastes three teams’ time.

Stage 2 · Target selection

Which repository the work belongs in, and which files inside it AI is even allowed to look at.

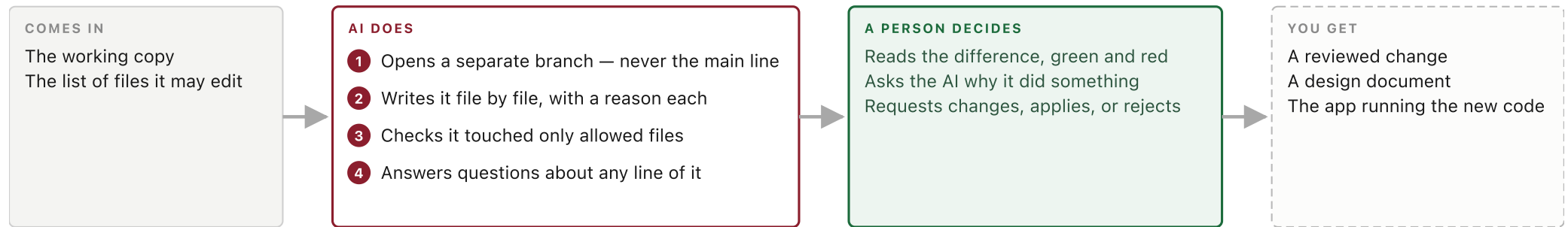


How the file list narrows before a single line is written

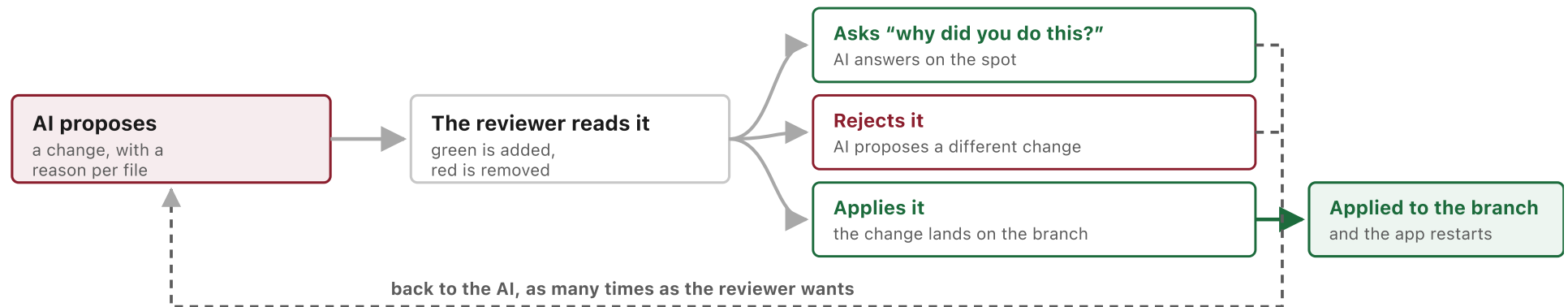


Stage 3 · Code generation

The change itself — written on a branch, explained line by line, and applied only when a person agrees.

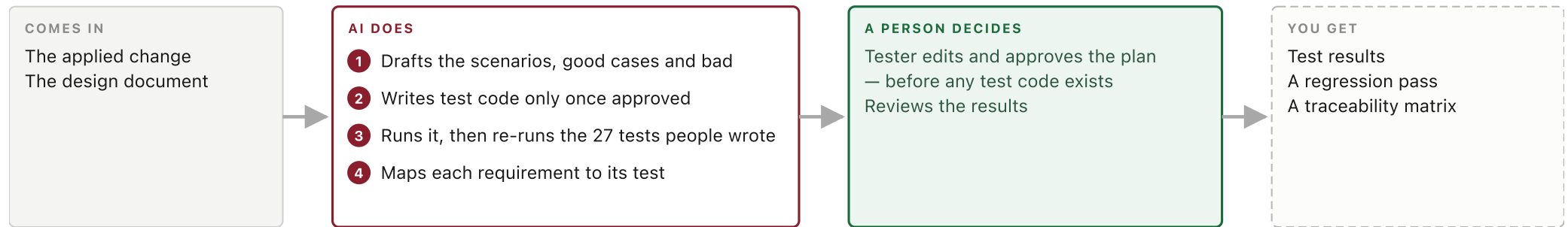


The review loop — nothing reaches the repository until a person says so

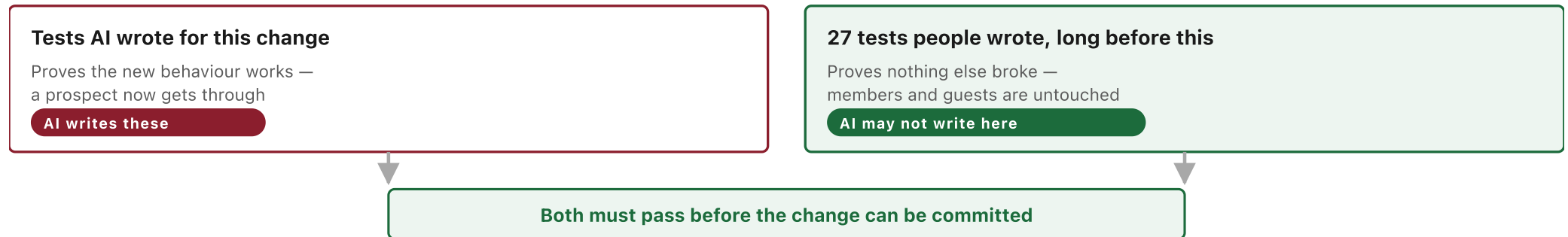


Stage 4 · Test

Proving the new behaviour works, and — separately — proving nothing else broke.



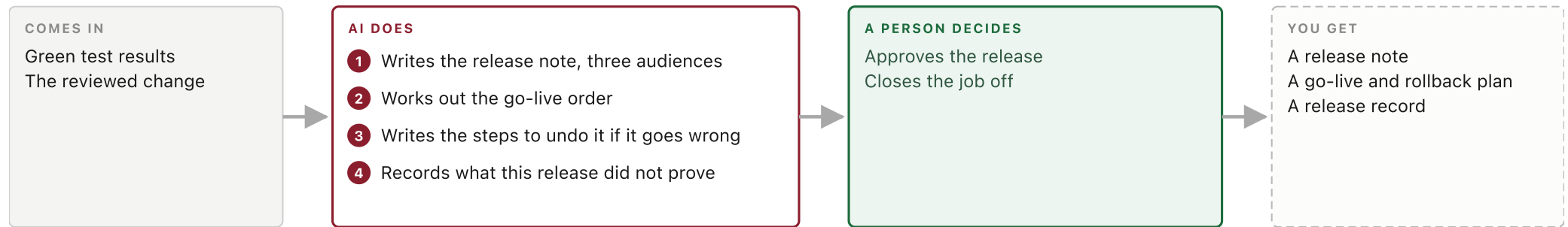
Two suites — and AI is allowed to write only one of them



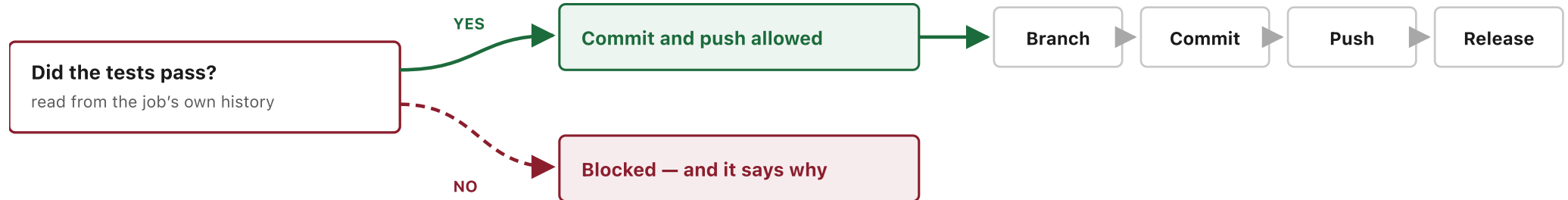
A bug is then put into the code on purpose, to prove these tests would actually have caught it. Passing tests that catch nothing are worse than none.

Stage 5 · Release

Everything needed to put it live, assembled from what the run actually did.



The gate before anything is committed



The answer is read from the job's own history — never from the screen. A screen that could claim "tests passed" could commit a broken change.

What this actually gives you

4 OF 9 FILES



1 OF 5 SYSTEMS



27 ALL PASSING



5 SIGN-OFFS



Points where a person has to say yes before anything moves on.

How the work changes

TODAY

Gaps in the request show up late, often after code is written



WITH AI HELPING

The questions get asked before anything is written

Other teams find out late, or get told about work they don't have



Only the team that really has work gets a job raised

A reviewer reads the code and guesses why it was written that way



The reviewer asks "why did you do this?" and gets an answer

Older tests get re-run separately, if there is time



They are re-run every time, as part of the same step

Release paperwork is written at the end, from memory



It is built from what the run actually did

What AI is not allowed to do

- **Touch the older tests.** People wrote them. They are the independent check that nothing else broke — so AI cannot edit them to agree with itself.
- **Mark its own homework.** Whether the change is good to go is read from the job's own history, not from anything AI claims.
- **Put anything live on its own.** Everything sits on a separate copy until a person approves the release.
- **Automate everything.** Some tests still have to be done by hand — mainframe and other systems AI cannot reach. Those are listed as manual, not hidden.